# Informe Laboratorio 4

**Sección 1**

Darien Reyes
e-mail: darien.reyes@mail.udp.cl

Noviembre de 2026

# Índice

**1. Descripción** **2**

**2. Actividades** **2**
2.1. Configuración y análisis de SSH . . . 2
2.1.1. Implementación y configuración del servidor SSH . . . 2
2.1.2. Análisis de paquetes SSH en Wireshark . . . 2
2.2. Análisis de captura de tráfico antigua . . . 3
2.2.1. Análisis de los mecanismos criptográficos . . . 3
2.2.2. Recuperación del PIN y derivación de la clave AES . . . 3
2.2.3. Descifrado de las transacciones y análisis final . . . 3

**3. Desarrollo de actividades** **5**
3.1. Actividad 1 . . . 5
3.1.1. Dockerfile . . . 5
3.1.2. Configuración del servidor . . . 5
3.1.3. Explicación de los algoritmos utilizados . . . 6
3.1.4. Conexión cliente-servidor . . . 6
3.1.5. Análisis de tráfico . . . 7
3.2. Actividad 2 . . . 8
3.2.1. Identificación del paquete RSA_PIN . . . 8
3.2.2. Identificación de los paquetes AESGCM . . . 8
3.2.3. Recuperación del PIN y derivación de la clave AES . . . 9
3.2.4. Descifrar los mensajes . . . 9

**4. Bibliografia** **13**

# 1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

# 2. Actividades

## 2.1. Configuración y análisis de SSH

### 2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

### 2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

## 2.2. Análisis de captura de tráfico antigua

### 2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato .pcap, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

### 2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta RSA_PIN, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: $(n = 2^{14})$, $(r = 8)$ y $(p = 1)$. Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

### 2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.

actividades/Screenshot 2026-10-15 164449.png

Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

# 3. Desarrollo de actividades

## 3.1. Actividad 1

### 3.1.1. Dockerfile

Se construyó una imagen de Docker personalizada basada en Ubuntu que instala y ejecuta un servidor OpenSSH. Durante la construcción, se eliminaron las claves por defecto para generar únicamente la clave ED25519 requerida, y se copió el archivo de configuración personalizado.

```
darien@darien-Aspire-A514-53:~/lab4_ssh$ cat Dockerfile
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y openssh-server && rm -rf /var/lib/apt/lists/*

RUN mkdir -p /var/run/sshd

RUN rm -f /etc/ssh/ssh_host_* && \
    ssh-keygen -t ed25519 -f /etc/ssh/ssh_host_ed25519_key -N ""

RUN useradd -rm -d /home/labuser -s /bin/bash -g root -G sudo -u 1000 labuser && \
    echo 'labuser:password123' | chpasswd

COPY sshd_config /etc/ssh/sshd_config

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
darien@darien-Aspire-A514-53:~/lab4_ssh$
```

Figura 2: Archivo Dockerfile y verificación del contenedor en ejecución.

### 3.1.2. Configuración del servidor

El servidor fue configurado exitosamente mediante el archivo `sshd_config` para aceptar exclusivamente los algoritmos criptográficos requeridos por el laboratorio. Esto se verifica en la captura de red, donde el servidor expone su lista restringida de algoritmos.

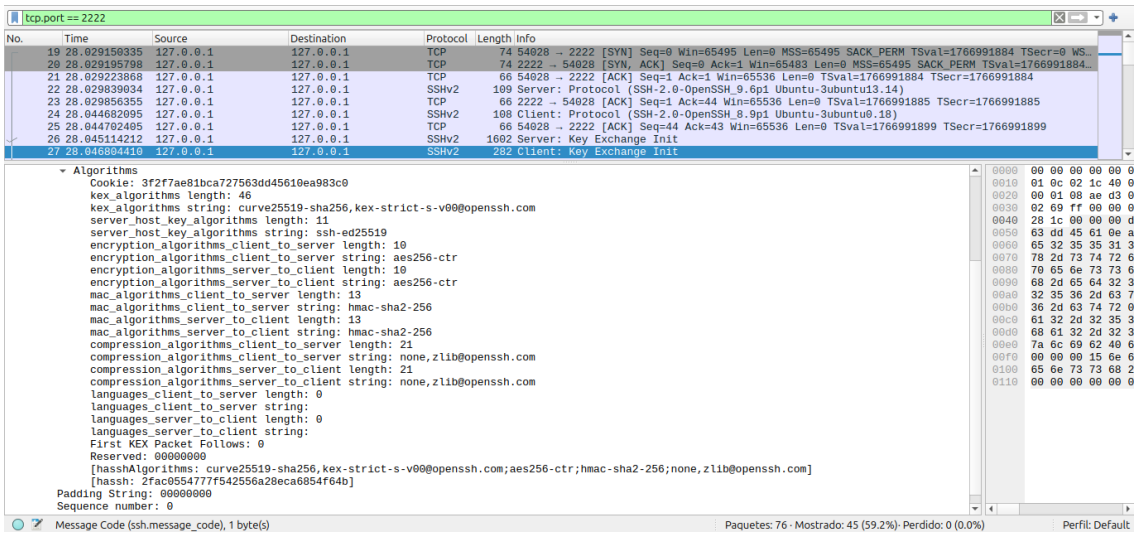



Figura 3: Paquete de respuesta del servidor mostrando los algoritmos configurados.

### 3.1.3. Explicación de los algoritmos utilizados

- `curve25519-sha256` **(Intercambio de claves - KEX):** Protocolo basado en criptografía de curva elíptica que permite a cliente y servidor acordar un secreto compartido de forma segura sobre un canal inseguro. Provee secreto hacia adelante (*forward secrecy*).

- `aes256-ctr` **(Cifrado simétrico - Cipher):** Algoritmo de cifrado en bloque configurado en modo contador (CTR). Su función es cifrar el tráfico y los datos de la sesión para garantizar la confidencialidad.

- `hmac-sha2-256` **(Autenticación de mensajes - MAC):** Mecanismo que genera una etiqueta criptográfica mediante funciones hash para verificar la integridad de los datos, asegurando que los paquetes no sean alterados en tránsito.

- `ssh-ed25519` **(Clave de Host):** Esquema de firma digital de clave pública basado en curvas elípticas. Permite la autenticación del servidor, protegiendo al cliente contra ataques de intermediario (*Man-in-the-Middle*).

### 3.1.4. Conexión cliente-servidor

Se estableció una conexión SSH funcional desde la máquina anfitriona hacia el contenedor a través del puerto mapeado (2222), verificando exitosamente la huella digital del host.

```
darien@darien-Aspire-A514-53:~$ ssh -p 2222 labuser@127.0.0.1
The authenticity of host '[127.0.0.1]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:UQqWmm8wW039zhgHa5agYrxnoiSF6gje9R21pbdzIB8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[127.0.0.1]:2222' (ED25519) to the list of known hosts.
labuser@127.0.0.1's password:
labuser@3490b20a337a:~$
```

Figura 4: Conexión exitosa al contenedor SSH.

### 3.1.5. Análisis de tráfico

A partir de la captura en Wireshark, es posible identificar los siguientes tres elementos antes de que la sesión sea cifrada:

1. **Intercambio de versiones del protocolo:** Los paquetes iniciales se transmiten en texto plano, revelando la versión del software del cliente y del servidor (ej. `SSH-2.0-OpenSSH_8.9p1`)
2. **Negociación de algoritmos (`SSH_MSG_KEXINIT`):** Ambos extremos publican en claro las suites criptográficas que soportan, permitiendo a un tercero observar los mecanismos de seguridad propuestos.
3. **Transición a tráfico cifrado:** Tras el intercambio de claves públicas, se envía el mensaje `New Keys`. A partir de este momento, todos los paquetes subsecuentes se visualizan como `Encrypted packet` y su contenido es ininteligible.

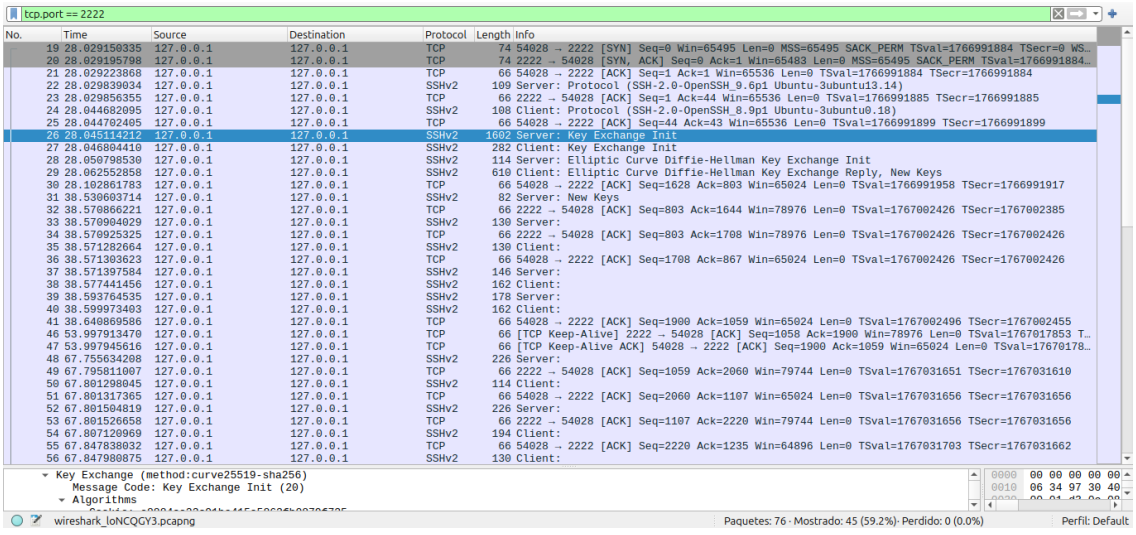



Figura 5: Captura del tráfico de negociación SSH en Wireshark.

## 3.2. Actividad 2

### 3.2.1. Identificación del paquete RSA_PIN

Se analizó la captura `.pcap` provista. El primer paquete relevante (Paquete #1) contiene una carga útil de 280 bytes correspondiente a la etiqueta `RSA_PIN`, seguida de 16 bytes de sal y 256 bytes de datos cifrados con RSA.

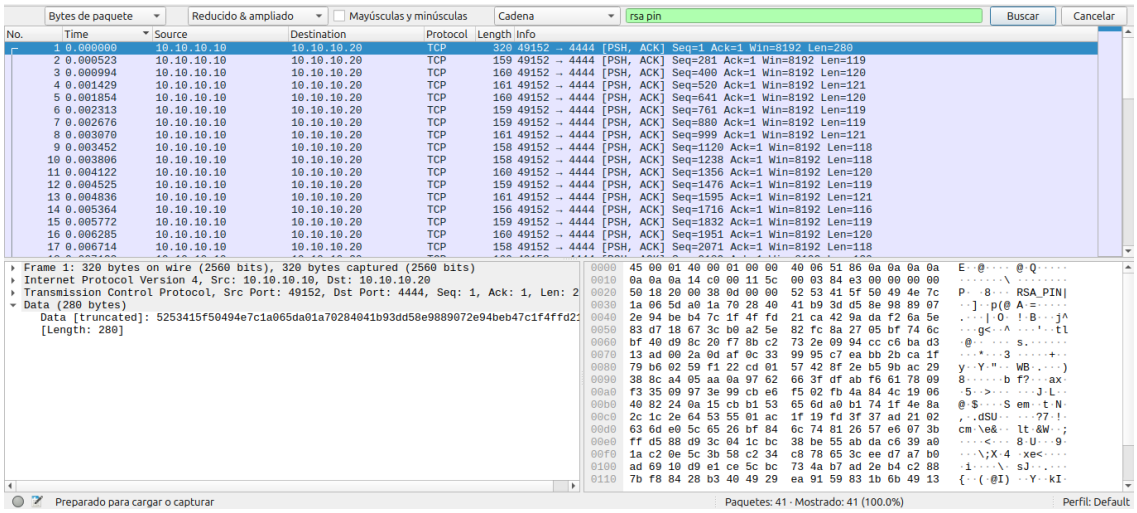



Figura 6: Paquete #1 conteniendo el PIN cifrado y la salt.

### 3.2.2. Identificación de los paquetes AESGCM

Los paquetes subsecuentes (del 2 al 41) corresponden a las transacciones financieras. Cada uno posee una longitud de datos reducida ( 119-121 bytes) compuesta por el *nonce* de 12 bytes y el texto cifrado con AES-GCM.

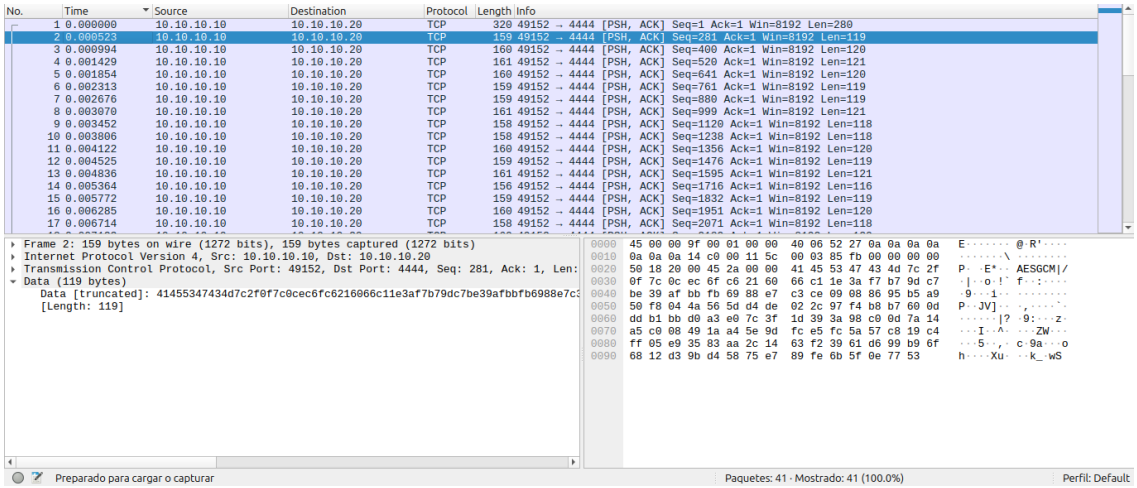



Figura 7: Identificación de los paquetes de transacciones financieras.

### 3.2.3. Recuperación del PIN y derivación de la clave AES

Se implementó un script que extrae la sal y descifra el bloque RSA-OAEP utilizando la clave privada suministrada, obteniendo el PIN original. Posteriormente, se derivó la clave AES de 256 bits mediante Scrypt.

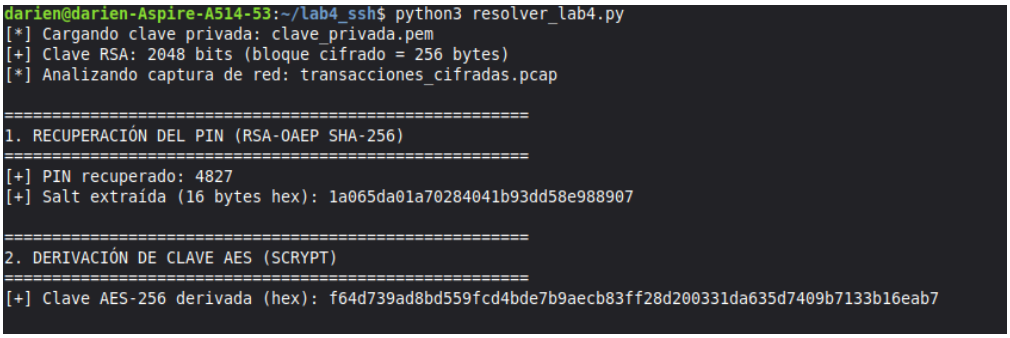



Figura 8: Recuperación del PIN y derivación de clave en consola.

**Función de la *salt*:** Un PIN de cuatro dígitos tiene solo 10.000 combinaciones posibles. La sal añade 16 bytes de entropía aleatoria a la entrada de la función de derivación, lo que previene ataques precomputados masivos (como *rainbow tables*) y obliga al atacante a calcular las derivaciones desde cero para cada sesión.

**Necesidad de parámetros idénticos ($n = 2^{14}, r = 8, p = 1$):** Scrypt es una función *memory-hard* determinista diseñada para ser costosa computacionalmente. Los parámetros controlan el costo de CPU/memoria y la paralelización. Si no se utilizan exactamente los mismos valores configurados por el emisor, las operaciones algebraicas divergen por completo, generando una clave AES-256 totalmente distinta e inútil para el descifrado.

### 3.2.4. Descifrar los mensajes

Una vez obtenida la clave AES-GCM, el script aisló el *nonce* de cada paquete y recuperó los 40 mensajes de transacciones en texto plano, validando su autenticidad.

```
======================================================================
         3. TRANSACCIONES FINANCIERAS RECUPERADAS (AES-GCM)
======================================================================
[Paquete #02] TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923
[Paquete #03] TRANSACCION COMPLETADA|ID=TRX-346837|DE=Catalina Vargas|A=Valentina Rojas|MONTO=58911
[Paquete #04] TRANSACCION COMPLETADA|ID=TRX-412154|DE=Sebastian Munoz|A=Constanza Vidal|MONTO=485213
[Paquete #05] TRANSACCION COMPLETADA|ID=TRX-071484|DE=Felipe Morales|A=Tomas Sepulveda|MONTO=314718
[Paquete #06] TRANSACCION COMPLETADA|ID=TRX-805221|DE=Isidora Silva|A=Javiera Fuentes|MONTO=104470
[Paquete #07] TRANSACCION COMPLETADA|ID=TRX-971697|DE=Valentina Rojas|A=Camila Torres|MONTO=785714
[Paquete #08] TRANSACCION COMPLETADA|ID=TRX-139273|DE=Catalina Vargas|A=Nicolas Herrera|MONTO=913757
[Paquete #09] TRANSACCION COMPLETADA|ID=TRX-652114|DE=Camila Torres|A=Fernanda Araya|MONTO=855548
[Paquete #10] TRANSACCION COMPLETADA|ID=TRX-432361|DE=Vicente Soto|A=Matias Gonzalez|MONTO=818686
[Paquete #11] TRANSACCION COMPLETADA|ID=TRX-818929|DE=Antonia Castro|A=Tomas Sepulveda|MONTO=885868
[Paquete #12] TRANSACCION COMPLETADA|ID=TRX-871181|DE=Catalina Vargas|A=Camila Torres|MONTO=514444
[Paquete #13] TRANSACCION COMPLETADA|ID=TRX-157566|DE=Tomas Sepulveda|A=Javiera Fuentes|MONTO=436301
[Paquete #14] TRANSACCION COMPLETADA|ID=TRX-186243|DE=Vicente Soto|A=Daniela Reyes|MONTO=698110
[Paquete #15] TRANSACCION COMPLETADA|ID=TRX-121024|DE=Isidora Silva|A=Sebastian Munoz|MONTO=623481
[Paquete #16] TRANSACCION COMPLETADA|ID=TRX-339097|DE=Matias Gonzalez|A=Constanza Vidal|MONTO=77226
[Paquete #17] TRANSACCION COMPLETADA|ID=TRX-663549|DE=Isidora Silva|A=Antonia Castro|MONTO=576901
[Paquete #18] TRANSACCION COMPLETADA|ID=TRX-367001|DE=Fernanda Araya|A=Tomas Sepulveda|MONTO=282567
[Paquete #19] TRANSACCION COMPLETADA|ID=TRX-188807|DE=Camila Torres|A=Antonia Castro|MONTO=195173
[Paquete #20] TRANSACCION COMPLETADA|ID=TRX-096936|DE=Felipe Morales|A=Antonia Castro|MONTO=359917
[Paquete #21] TRANSACCION COMPLETADA|ID=TRX-695623|DE=Catalina Vargas|A=Nicolas Herrera|MONTO=619299
[Paquete #22] TRANSACCION COMPLETADA|ID=TRX-407749|DE=Camila Torres|A=Sebastian Munoz|MONTO=182369
[Paquete #23] TRANSACCION COMPLETADA|ID=TRX-069658|DE=Valentina Rojas|A=Javiera Fuentes|MONTO=556520
[Paquete #24] TRANSACCION COMPLETADA|ID=TRX-232469|DE=Felipe Morales|A=Tomas Sepulveda|MONTO=23769
[Paquete #25] TRANSACCION COMPLETADA|ID=TRX-940466|DE=Vicente Soto|A=Fernanda Araya|MONTO=292293
[Paquete #26] TRANSACCION COMPLETADA|ID=TRX-755002|DE=Matias Gonzalez|A=Daniela Reyes|MONTO=946989
[Paquete #27] TRANSACCION COMPLETADA|ID=TRX-962321|DE=Felipe Morales|A=Daniela Reyes|MONTO=589780
[Paquete #28] TRANSACCION COMPLETADA|ID=TRX-324981|DE=Diego Ramirez|A=Daniela Reyes|MONTO=971045
[Paquete #29] TRANSACCION COMPLETADA|ID=TRX-337011|DE=Antonia Castro|A=Felipe Morales|MONTO=777198
[Paquete #30] TRANSACCION COMPLETADA|ID=TRX-995006|DE=Vicente Soto|A=Sebastian Munoz|MONTO=587590
[Paquete #31] TRANSACCION COMPLETADA|ID=TRX-630572|DE=Camila Torres|A=Sebastian Munoz|MONTO=332702
[Paquete #32] TRANSACCION COMPLETADA|ID=TRX-517969|DE=Catalina Vargas|A=Daniela Reyes|MONTO=484770
[Paquete #33] TRANSACCION COMPLETADA|ID=TRX-447236|DE=Matias Gonzalez|A=Constanza Vidal|MONTO=331352
[Paquete #34] TRANSACCION COMPLETADA|ID=TRX-852645|DE=Constanza Vidal|A=Felipe Morales|MONTO=343065
[Paquete #35] TRANSACCION COMPLETADA|ID=TRX-188241|DE=Camila Torres|A=Constanza Vidal|MONTO=781107
[Paquete #36] TRANSACCION COMPLETADA|ID=TRX-713923|DE=Diego Ramirez|A=Catalina Vargas|MONTO=21358
[Paquete #37] TRANSACCION COMPLETADA|ID=TRX-362125|DE=Camila Torres|A=Sebastian Munoz|MONTO=154118
```

Figura 9: Listado de las 40 transacciones recuperadas.

**Código del Script (Python):**

```python
import sys
from scapy.all import rdpcap, Raw
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

PCAP_FILE = "transacciones_cifradas.pcap"
KEY_FILE = "clave_privada.pem"

print(f"[*] Cargando clave privada: {KEY_FILE}")
with open(KEY_FILE, "rb") as f:
    private_key = serialization.load_pem_private_key(f.read(), password=
    None)

rsa_bytes_len = private_key.key_size // 8
print(f"[+] Clave RSA: {private_key.key_size} bits (bloque = {
    rsa_bytes_len} bytes)")

print(f"[*] Analizando captura de red: {PCAP_FILE}")
```

```
paquetes = rdpcap(PCAP_FILE)

pkt1_data = bytes(paquetes[0][Raw].load)

if b"|" in pkt1_data[:15]:
    datos_pin = pkt1_data.split(b"|", 1)[1]
else:
    datos_pin = pkt1_data

salt = datos_pin[:16]
bloque_rsa = datos_pin[16:16 + rsa_bytes_len]

print("\n" + "="*55)
print("1. RECUPERACION DEL PIN (RSA-OAEP SHA-256)")
print("="*55)

try:
    pin_bytes = private_key.decrypt(
        bloque_rsa,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    pin_str = pin_bytes.decode('utf-8', errors='ignore').strip()
    print(f"[+] PIN recuperado: {pin_str}")
    print(f"[+] Salt extra da (16 bytes hex): {salt.hex()}")
except Exception as e:
    print(f"[-] Error al descifrar el bloque RSA: {e}")
    sys.exit(1)

print("\n" + "="*55)
print("2. DERIVACI N DE CLAVE AES (SCRYPT)")
print("="*55)

kdf = Scrypt(
    salt=salt,
    length=32,
    n=2**14,
    r=8,
    p=1
)
clave_aes = kdf.derive(pin_str.encode('utf-8'))
print(f"[+] Clave AES-256 derivada (hex): {clave_aes.hex()}")

print("\n" + "="*70)
print(f"{'3. TRANSACCIONES FINANCIERAS RECUPERADAS (AES-GCM)':^70}")
print("="*70)

aesgcm = AESGCM(clave_aes)
exitos = 0
```

```
71
72 for i, pkt in enumerate(paquetes[1:], start=2):
73     if Raw in pkt:
74         payload = bytes(pkt[Raw].load)
75
76         if b"|" in payload[:15]:
77             datos_cifrados = payload.split(b"|", 1)[1]
78         else:
79             datos_cifrados = payload
80
81         if len(datos_cifrados) < 28:
82             continue
83
84         nonce = datos_cifrados[:12]
85         ciphertext = datos_cifrados[12:]
86
87         try:
88             plaintext = aesgcm.decrypt(nonce, ciphertext, None)
89             mensaje = plaintext.decode('utf-8', errors='ignore').strip()
90             print(f"[Paquete #{i:02d}] {mensaje}")
91             exitos += 1
92         except Exception:
93             pass
94
95 if exitos == 0:
96     kdf_fallback = Scrypt(salt=salt, length=32, n=2**14, r=8, p=1)
97     clave_aes_alt = kdf_fallback.derive(pin_bytes)
98     aesgcm_alt = AESGCM(clave_aes_alt)
99     for i, pkt in enumerate(paquetes[1:], start=2):
100         if Raw in pkt:
101             payload = bytes(pkt[Raw].load)
102             datos_cifrados = payload.split(b"|", 1)[1] if b"|" in payload
    [:15] else payload
103             if len(datos_cifrados) < 28:
104                 continue
105             nonce = datos_cifrados[:12]
106             ciphertext = datos_cifrados[12:]
107             try:
108                 plaintext = aesgcm_alt.decrypt(nonce, ciphertext, None)
109                 mensaje = plaintext.decode('utf-8', errors='ignore').strip
    ()
110                 print(f"[Paquete #{i:02d}] {mensaje}")
111                 exitos += 1
112             except Exception:
113                 pass
114
115 print("="*70)
116 print(f"[+] Total de transacciones descifradas con  xito : {exitos}")
```

## Conclusiones y comentarios

**Sobre la Criptografía Post-cuántica (PQC):** Frente a la interrogante de si sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC, la respuesta es **no**. Los algoritmos postcuánticos (como ML-KEM/Kyber) basan su seguridad en problemas algebraicos intratables para computadores cuánticos, tales como el Aprendizaje con Errores sobre Retículos (M-LWE). Por el contrario, RSA basa su seguridad en la factorización de enteros compuestos ($N = p \cdot q$). Las claves RSA carecen de compatibilidad matemática, algebraica y dimensional con las operaciones requeridas por primitivas PQC, volviendo imposible su uso para descifrar datos protegidos bajo este nuevo paradigma.

**Comentarios generales:** El desarrollo de este laboratorio permitió evidenciar de manera práctica el funcionamiento de la criptografía híbrida moderna. A través de la configuración de OpenSSH y el análisis en Wireshark, se demostró que un atacante pasivo solo puede observar la negociación inicial en claro antes de que el cifrado simétrico proteja la sesión. Asimismo, la recuperación del PIN y de las transacciones demostró la importancia crítica de aplicar sales criptográficas y funciones *memory-hard* (Scrypt) para evitar la rotura por fuerza bruta de secretos pequeños.

# 4. Bibliografia


- Captura y Claves proporcionadas para el desarrollo del laboratorio.